We thank the reviewers for their positive comments and insightful suggestions. We first answer the main questions, then provide tables for the additional evaluation results and finally provide answers to inline questions/comments.

Main Questions

[Reviewer A]

> Q1. In the rule generation phase (Section 3.1), are only API signatures passed into LLMs? Why is the document not passed, which is a natural source of such constraint description?

We do provide API documentation along with the grammar, error messages, and example rules. We will clarify this in the paper.

> Q2. In Algorithm 1, line 5, how many error messages are included in the prompt?

The number of error messages included in the prompt varies per API. We dynamically collected error messages by running all APIs with invalid inputs, generated by applying nine kinds of mutations on LLM-generated valid inputs and random generation. In total, we use 5,667 error messages, and the average number per API is 2.54. We will clarify this in the paper.

> Q3. Section 3.1.2 mentions an iterative query strategy. On average, how many iterations does it take per API?

The exact number of iterations varies across APIs. Given a one-minute time budget, the average number of iterations is 18.4. Notably, for more efficient rule generation, we ask the LLM to return multiple rules in a single response. This enables the generation of 93.37 rules on average (max: 213) within a minute. We will clarify this in Section 3.1.2.

> Q4. In line 579, are the heuristics generalizable and fully automated?

Yes, the heuristics (to refine constraints based on validity ratio) can be generally applied to any libraries/APIs and are fully automated.

> Q5. In Figure 5, the authors define a grammar for API constraints. Could the authors discuss the limitations of its expressiveness? Does it capture all possible program properties? If not, what kinds of cases cannot be expressed? For instance, nested structures or complex shape constraints such as `a.shape[0] = b`?

We believe Centaur's grammar is expressive enough to capture most properties. We carefully designed the grammar to express (1) all the common properties of API parameters (e.g., tensors), (2) relational properties in first-order logic, (3) common nested data structures such as tuples and lists, and (4) complex types like unions. Our grammar can also express `a.shape[0] = b` (concretely as `shape(a, 0) = b`). Indeed, there are properties that can't be expressed in the grammar, e.g., properties conditioned on the existence of optional parameters. One example is: "if parameter `axis` exists, then `axis` must be in the range of `[-ndim(input), ndim(input)-1]`" for the `tf.reduce_sum` API. However, Centaur checks the API signature and learns the constraints only when the optional argument exists. Also, our grammar definition is based on the Lark format and can be easily extended. We will add this discussion to the paper.

> Q6.1 What is the bug detection procedure? The paper mentions that two types of bugs are considered: crashes and inconsistencies. However, Table 6 also reports two additional types: NaN and overflow. How are these two categories detected and flagged?

We use two primary oracles for bug detection: crashes and CPU vs. GPU output inconsistencies. After manual verification, the inconsistencies are further classified into three categories: (1) NaN: The inconsistency is caused by a Not a Number value generated on only one platform, (2) Overflow: The inconsistency stems from an arithmetic overflow occurring on only one platform, and (3) Inconsistent: The difference is due to a logical flaw in an implementation, not a NaN or overflow.

> Q6.2 Furthermore, how are potential false positives handled?

To ensure the integrity of our results, every potential bug flagged by our oracles undergoes a rigorous manual verification process. This step is essential for filtering out false positives, such as minor, acceptable floating-point precision differences that might be flagged by our inconsistency oracle. For every bug that we confirm as a true positive, we create a minimized, reproducible test case. This ensures that all reported bugs are genuine and clearly demonstrate the underlying fault.

[Reviewer B]

> Q1. What is the fundamental difference between the constraint learning in this work and the invariant learning in existing work (such as in the Daikon system)?

Our approach for constraint learning is indeed inspired by Daikon's dynamic invariant learning. In some sense, our constraints are similar to function preconditions learned by Daikon. However, Daikon validates a set of general (template-based) properties against a fixed set of inputs/traces, whereas Centaur uses an LLM to generate domain-specific candidate properties that are validated against a dynamically generated set of inputs (using specialized mutations, random values, etc.). This approach allows Centaur to learn very precise, complete, and domain-relevant properties (as demonstrated by our results). Finally, unlike Daikon, Centaur includes a rule refinement process that eliminates redundant or spurious rules.

> Q2. How dependent is rule quality on the specific LLM used, and what challenges would arise if a different LLM provides lower-quality rule candidates?

Our rule generation quality depends on the quality of the model being used. However, conceptually, Centaur can still generate high-quality constraints as it (1) provides five types of feedback to effectively guide the LLM, (2) type checks the constraints, and (3) uses valid inputs to remove low-quality candidate rules. We agree that it is important to evaluate the impact of LLM choice. Table 1 (below) shows the results of our limited study on different LLMs with 12 randomly selected PyTorch APIs. We consider GPT-5 as another closed-source LLM (besides gemini-flash, our default LLM) and two variants of the open source Gemma 3.0 - 27b (large) and 12b (small) to represent small and large models. The open source models fail to execute some APIs due to poorer quality seed inputs. This can be alleviated by tweaking the number of times a feedback is provided to the LLM to get a set of valid inputs, if one wishes to do so. However, branch coverage and validity ratio is excellent across all LLMs. These initial results suggest that Centaur can work with different LLMs, including open-source (and smaller) LLMs. We will elaborate on these results in the final version of the paper.

> Q3. Is validating with a small set of seed inputs sufficient to mitigate risks from LLM "hallucinations" or missed implicit constraints?

Yes, if the number of seed inputs is small and there is a chance that wrong candidates are pruned less effectively. To mitigate this, we first obtain a small number of valid inputs using an LLM and apply nine types of mutations to enrich the set. We carefully designed these mutators to modify inputs in valid ways while increasing diversity. In the end, we use about 117 valid inputs on average per API. We will add this to the paper.

> Q4. Did the authors repeat the experiments multiple times to mitigate the randomness in the results generated by the LLM?

Although we used a ruleset generated from a single run for evaluation, we believe the impact of randomness is minimized because we (1) use an iterative process to obtain candidate rules, and (2) apply the process across all APIs of both libraries. Also, during implementation, we ran the rule generation multiple times and did not observe any significant differences in quality.

[Reviewer C]

> Q1. Please report how often the refinement step discards correct constraints. Could this limit bug discovery?

During this rebuttal period, we conducted a limited study on five widely used APIs: `torch.argmax`, `torch.nn.functional.alpha_dropout`, `torch.nn.ReLU`, `tf.math.sigmoid`, and `tf.image.transpose`. Centaur initially generated 141 constraints in total and filtered out 119 based on validity ratio. We manually examined the filtered constraints and confirmed that all were correctly excluded. In particular, they fell into three categories: (1) duplicates, (2) default constraints provided by Centaur (e.g., parameter types), and (3) incorrect constraints. However, we acknowledge that Centaur could potentially discard correct constraints during refinement. We will include this discussion in the Threats to Validity section of our final draft.

> Q2. Please clarify why statistical tests are applied to coverage but not to validity or bug counts.

The initial decision to omit the tests for validity ratio was due to the consistently high ratio for Centaur, which is near 100% across all comparisons. That being said, we present Table 2 (below), which shows the tests of statistical significance and effect sizes (as in coverage). Results show the superior performance of Centaur compared to all other state-of-the-art tools. For bugs, the tests were not applied as it requires substantially more effort to compare. For the same reason, prior work (e.g., TitanFuzz) did not include side comparisons.

Result Tables

Table 1: Comparison between using different LLMs with 12 **PyTorch** APIs

LLM	# APIs Successfully Executed	Avg. # Branches Covered	Avg. Validity Ratio (%)
Gemini Flash 2.0	12/12	9,772.67	92.48%
GPT - 5	12/12	9,750.83	91.20%
Gemma 3.0 (27b)	8/12	9,584.50	88.67%
Gemma 3.0 (12b)	9/12	9,847.78	87.99%

Table 2: Statistical test for validity ratio between Centaur and SoTA techniques

Library	vs Tool	# APIs	T-test	Cohen's D	Effect Size
PyTorch	Titanfuzz	211	2.47E-139	3.93	Large
	ACETest	136	1.60E-27	1.6	Large
	Pathfinder	217	1.72E-77	2.4	Large
TensorFlow	Titanfuzz	334	0	6.42	Large
	ACETest	151	3.32E-19	1.11	Large
	Pathfinder	157	1.70E-14	0.91	Large

Table 3: Ablation study of Centaur with and without rule refinement on 50 APIs per library

Library	Ablation	Avg. # Branches Covered	Avg. Validity Ratio (%)
PyTorch	Centaur	9930.86	100.00%
	w/o Rule Refinement	9826.64	98.44%
TensorFlow	Centaur	4164.26	93.78%
	w/o Rule Refinement	4103.82	86.86%

Table 4: Comparison with SoTA with a 10 minute budget for 10 PyTorch APIs

Tool	Avg. # Branches Covered	Avg. Validity Ratio (%)
Pathfinder	2,803.50	56.50%
Centaur	9,990.80	97.02%
ACETest	9,725.71	52.19%
Centaur	9,849.29	100.00%
Titanfuzz	9,501.00	22.94%
Centaur	9,962.00	100.00%

Table 5: Number of valid inputs generated by each tool (updated with improved results)

Tool	PyTorch	TensorFlow
Titanfuzz	113k	29k
Centaur	35,558k	35,987k
Improvement	35,446k	35,959k
ACETest	1,683k	9,804k
Centaur	26,990k	14,810k
Improvement	25,307k	5,005k
Pathfinder	5,500k	103k
Centaur	46,731k	14,366k
Improvement	41,231k	14,264k

Inline Questions/Comments

[Reviewers A,B]

> The manual evaluation in Section 5.1.3 is conducted on only 20 APIs, which is insufficient to draw reliable or representative conclusions

> The qualitative evaluation of learned constraints is based on a small sample of APIs

We agree that 20 APIs may be insufficient to draw a complete conclusion. During this rebuttal period, we analyzed the constraints for five more widely used APIs: `torch.argmax`, `torch.nn.functional.alpha_dropout`, `torch.nn.ReLU`, `tf.math.sigmoid`, and `tf.image.transpose`. Centaur generated a total of 22 constraints, and when compared with the 9 ground truth constraints that we manually wrote, it achieved 100% precision and recall. We would be happy to include these results, as well as analyze several more APIs that cover most library functionalities, for the qualitative analysis in our final draft.

[Reviewers A,C]

> While it is effective at removing incorrect rules, this objective could also lead to loose rules that accept all inputs.

> The authors observe this phenomenon only on a small sample of 10 APIs per library. There is no systematic measurement of how often pruning discards true constraints across all APIs.

Thanks for pointing this out. However, our manual analysis confirmed that all the constraints removed by the refinement were correctly excluded. Please check our answer to Q1 from Reviewer C for more details.

[Reviewer A]

> The paper is also missing a Threats to Validity section.

We agree that a Threats to Validity section is necessary. We will include the section in our final draft and discuss external and internal threats such as the potential limitation of the grammar, the possibility of false positives, and the reliability of our manual evaluation, including the impact of small sample sizes on the results.

[Reviewer B]

> The contribution of each component within the complex, multi-stage pipeline is not individually assessed. An ablation study is needed to dissect the performance gains from each part of the framework. For example, an analysis could be added to quantify the impact of the grammar-guided LLM versus an unguided one, or measure the effectiveness of the rule refinement step in isolation.

An ablation study on the grammar-guided LLM is unfortunately not feasible, as Centaur has a multi-stage pipeline. Without the LLM generating outputs in a parsable format, the next stage cannot proceed. However, we agree that an ablation study on refinement is important. We conducted a limited study on 50 APIs from each library to provide an answer on that. Table 3 (above) shows that Centaur improves the validity ratio and branch coverage with the refinement by 4.58% and 1.18%, respectively. We will be happy to include these results in our final draft.

> The evaluation relies on a single model (Google Gemini 2.0 Flash), which limits the generalizability of the findings.

Please check the answer to question Q2 from Reviewer B and Table 1 (above).

> What is the cost of using LLMs? I assume it is rather cheap, but it would be better if the authors can report the cost (i.e., token usage).

For the generation of API signatures and seed inputs, we used 130.29K tokens per API with Gemini 2.0 Flash. For candidate rule generation, we used 201.29K tokens per API with the same model. Based on the model's official pricing, this corresponds to an estimate of $0.16 per API. Also, the cost is proportional to the generation time and can be even reduced with a smaller time budget.

> Please provide justifications on the 180-second time budget. Did the authors follow the existing work's settings?

To balance the cost of running a total 1,034 APIs across all baselines, we limit the time budget to 180 seconds per API. Centaur can generate tens of millions of inputs in 180 seconds (e.g., due to bucketing and blocking optimizations), allowing it to achieve high coverage in a short period. In contrast, in the same time frame, the other approaches generate 100x or 1000x fewer (and less diverse) inputs, and hence achieve lower coverage. To demonstrate this, we present the total number of valid inputs generated by each tool in Table 5 (above). Notably, we have significantly improved the efficiency of Centaur after submission.

To further illustrate Centaur's advantage, we select 10 PyTorch APIs and run them for 10 minutes for all baselines. Table 4 (above) contains the detailed results, which demonstrate that even on a longer time period, Centaur can outperform all the other state-of-the-arts on validity ratio and coverage.